

NEURAL OPS

DevOps Interview Brain

Master Product, Learning, AI, Simulation, Architecture & Technical Specification

From the original interview-preparation concept to the current React + MUI + motion/3D product direction.

Document scope

This document consolidates the full product vision and implementation decisions established across the design process. It is intended to function as the master reference for product planning, UI/UX implementation, engineering architecture, AI architecture, content design, and future development.

Master Table of Contents

- 1 Phase 0 — Product Vision & Problem Definition
- 2 Phase 1 — Learning & Knowledge Foundation
- 3 Phase 2 — AI/RAG Architecture
- 4 Phase 3 — Interview Question & Scenario Engine
- 5 Phase 4 — Architecture Studio
- 6 Phase 5 — Production Incident Simulator & Chaos
- 7 Phase 6 — Hands-on Labs & Terminal
- 8 Phase 7 — Observability Studio
- 9 Phase 8 — CI/CD & DevSecOps Studio
- 10 Phase 9 — AI Interviewer, Resume & JD Intelligence
- 11 Phase 10 — Behavioral, Communication & Experience Intelligence
- 12 Phase 11 — Adaptive Learning & Readiness Engine
- 13 Phase 12 — Full Interview Day & Emergency Mode
- 14 Phase 13 — UI/UX Design System & Current Visual Direction
- 15 Phase 14 — Motion, 3D & Advanced Frontend Stack
- 16 Phase 15 — Navigation & Information Architecture
- 17 Phase 16 — Data Model & Evidence Model
- 18 Phase 17 — Recommended Production Architecture
- 19 Phase 18 — Implementation Roadmap
- 20 Phase 19 — Current Prototype Status
- 21 Phase 20 — Future Expansion / Advanced Capabilities
- 22 Appendix A — Complete Technology & Scenario Library
- 23 Appendix B — Scoring & Evaluation Framework
- 24 Appendix C — Product Principles / Non-Negotiables

Phase 0 — Product Vision & Problem Definition

Product: Neural Ops — DevOps Interview Brain

Purpose: An AI-powered, evidence-based DevOps/DevSecOps/SRE/Cloud/Platform Engineer interview-preparation platform that combines learning, recall, hands-on practice, troubleshooting, production simulation, architecture design, AI interviewing, resume interrogation, JD analysis, behavioral preparation, and full interview simulation into one continuous readiness system.

Primary user goal: The user is preparing for Senior DevOps / DevSecOps / SRE / Cloud / Platform interviews and wants to crack the interview, not merely read documentation.

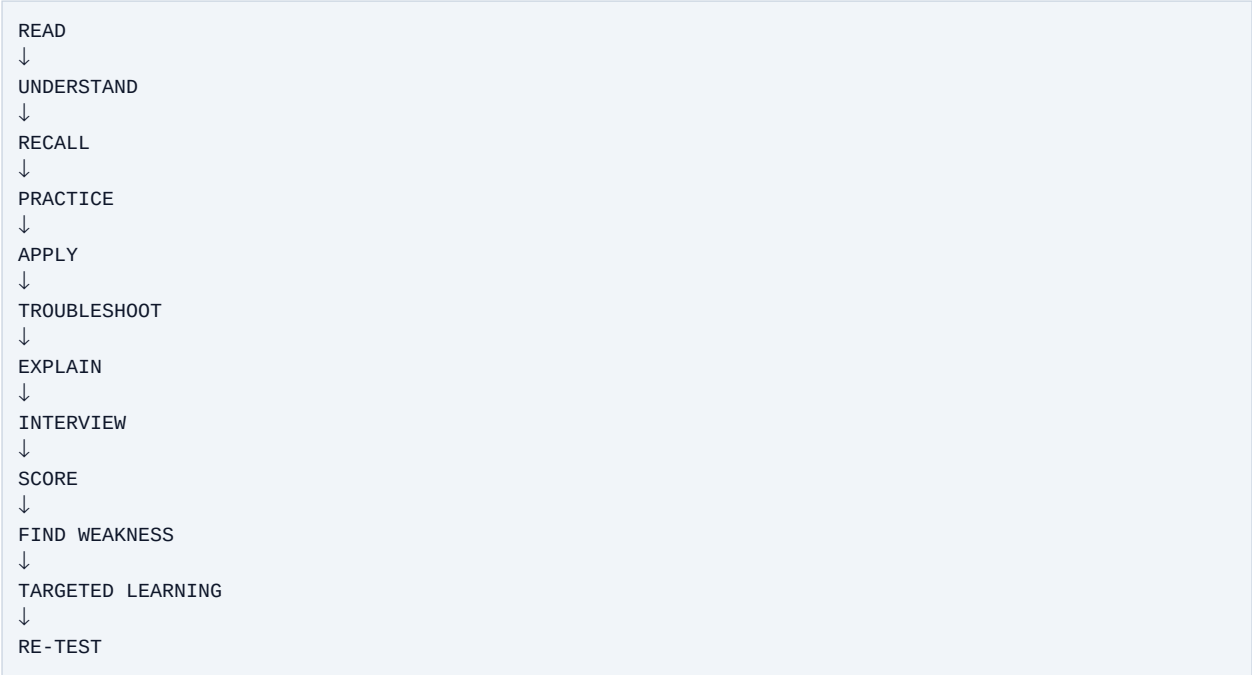
Core product question: “If I attend a Senior DevOps interview today, where will I fail, and what should I do today to fix it?”

The platform therefore treats interview readiness as an evidence problem. Reading is only one input. The system should collect evidence from recall, questions, labs, troubleshooting, architecture, incidents, communication and simulated interviews.

Five flagship capabilities

Capability	Purpose
Architecture Studio	Visually design architectures, validate requirements, receive deterministic + AI review, iterate, inject failures, and compare versions.
Production Simulator	Investigate realistic incidents using evidence, terminal commands and telemetry; mitigate, perform RCA, and propose prevention.
AI Interviewer	Run dynamic technical, architecture and behavioral interviews with resume/JD grounding and adaptive follow-ups.
Hands-on Labs	Prove practical ability through Linux, Docker, Kubernetes, Terraform, CI/CD and cloud-oriented operational tasks.
Readiness Engine	Continuously combine evidence into a Senior DevOps Interview Readiness Score and targeted daily plan.

Core learning loop



Core principle

A user must not be marked proficient simply because they read documentation. The system should require evidence of understanding, recall, application, troubleshooting and explanation.

Phase 1 — Learning & Knowledge Foundation

Knowledge Hub for every technology

- Learn — structured concepts and progressive learning paths.
- Documentation — official and personal source material with provenance.
- Mind Maps — visual relationships between concepts.
- Infographics — architecture and process explanations.
- Flashcards — recall-oriented learning with review history.
- Questions — knowledge, why, comparison, troubleshooting and interview questions.
- Commands — practical command references.
- Troubleshooting — guided diagnostic workflows.
- Incidents — production-style scenarios.
- Architecture — design patterns and system-design practice.
- Labs — hands-on operational tasks.
- Interview — technology-specific interview practice.
- Progress — evidence, weakness, confidence and mastery tracking.

Document ingestion

- Official documentation: AWS, Azure, Kubernetes, Docker, Terraform, Jenkins, ArgoCD, Helm, Ansible, Linux, Git, Python, PostgreSQL, Kafka, Redis, Prometheus, Grafana, networking and DevSecOps.
- Personal material: resume, project documentation, runbooks, architecture diagrams, troubleshooting notes, commands, interview notes and previous interview feedback.
- Job descriptions and company-specific material.
- Source provenance must be preserved so official source material is not confused with AI-generated interpretation.

Reading mode

- Full document reading.
- Select text and ask AI to explain, simplify, give examples, explain architecture, generate a question, create a flashcard, add a note or connect related topics.
- Context-aware AI assistance grounded in the selected source.

Mind maps and infographics

- Kubernetes → Control Plane / Networking / Storage / Security / Observability.
- CI/CD architecture and release flow.
- Terraform workflow.
- AWS networking.

- GitOps.
- Zero-downtime deployment.
- Disaster recovery.
- Microservices.

Flashcards and spaced repetition

- Concepts, commands, differences, architecture and troubleshooting facts.
- Track correct/incorrect, confidence, review history and weak cards.
- Use spaced repetition rather than simply repeating all cards equally.

Phase 2 — AI / RAG Architecture

Hybrid AI architecture

The agreed direction is a hybrid architecture. Deterministic software is authoritative for scoring, progress, thresholds, spaced repetition, question selection and readiness. Local LLMs provide cost-effective tutoring and generation; cloud LLMs are optional for difficult reasoning and high-quality evaluation.

```
LLMProvider
■■■ OllamaProvider
■■■ OpenAIProvider
■■■ AnthropicProvider
```

Recommended initial stack

- React + TypeScript
- FastAPI + Python
- PostgreSQL
- pgvector
- Ollama
- Redis
- Celery or equivalent task queue
- S3-compatible object storage
- Docker
- Kubernetes as the platform matures
- Prometheus + Grafana

AI responsibility split

Area	Recommended authority
Scoring rules	Deterministic engine
Readiness thresholds	Deterministic engine
Spaced repetition	Deterministic engine
Question selection	Deterministic + adaptive logic

Area	Recommended authority
RAG retrieval	Application + pgvector
Tutor / explanation	Local LLM by default
Summaries / flashcards	Local LLM
Scenario generation	Local LLM + validation
Basic interviewer	Local LLM
Complex architecture reasoning	Optional cloud LLM
Advanced answer evaluation	Cloud LLM optional
Voice evaluation	Cloud/local model depending capability

Evaluation model

LLMs should not blindly assign the final readiness score. Semantic evaluation can produce evidence dimensions, then deterministic weighting converts those dimensions into the authoritative score.

- Technical accuracy
- Completeness
- Troubleshooting method
- Communication
- Architecture reasoning
- Trade-off quality
- Evidence use
- Root-cause reasoning
- Remediation
- Prevention

Phase 3 — Interview Question & Scenario Engine

Question types

- Knowledge — e.g. What is Kubernetes Ingress?
- Why — e.g. Why use Terraform?
- Comparison — e.g. Terraform vs CloudFormation?
- Troubleshooting — e.g. A pod is CrashLoopBackOff. What do you do?
- Architecture — e.g. Design a highly available e-commerce platform.
- Scenario — e.g. Production DB connectivity suddenly fails.
- Trade-off — e.g. Why Kafka instead of SQS?
- Behavioral — e.g. Tell me about a production incident.
- Leadership — e.g. Tell me about disagreement with an architect.

Dynamic interviewer

The interviewer should follow the user's answer instead of presenting a static list. Each response influences the next question.

```
User:
"How would you troubleshoot CrashLoopBackOff?"

Interviewer:
"What would you check first?"

User:
"Logs."

Interviewer:
"What specifically would you look for, and what would you do
if the current container has no useful logs?"
```

Scenario engine

- Production Incident
- Architecture
- Scaling
- Failure
- Security
- Deployment
- Disaster Recovery
- Cost
- Troubleshooting
- Design Decision

Scenario graph

```
Initial Problem
↓
Expected Investigation
↓
Possible Paths
↓
Evidence
↓
Root Cause
↓
Remediation
↓
Prevention
↓
Follow-up Questions
```

The candidate's path should influence which evidence and follow-ups are revealed. This makes the interview adaptive rather than scripted.

Difficulty

Level	Meaning
L1	Basic
L2	Intermediate

Level	Meaning
L3	Senior
L4	Staff / Architect

Important interview behavior

The system should reward clarifying questions. It should not reward immediately naming a technology or remediation without establishing scope, symptoms, timeline, impact and recent changes.

Phase 4 — Architecture Studio

Architecture Studio is a flagship capability where the candidate designs a system visually, receives review, improves it, and demonstrates how the architecture behaves under failure.

Flow

```

Scenario
↓
Requirements
↓
Visual Architecture Builder
↓
Your Architecture
↓
Submit for Review
↓
AI + Rules Engine
↓
Architecture Review
↓
Scorecard
↓
Weaknesses + Recommendations
↓
Improve Architecture
↓
Re-submit

```

Requirements-first design

- Users
- Traffic
- Peak traffic
- Availability
- Latency
- Data volume
- Consistency
- Security
- Compliance
- RPO
- RTO
- Budget

- Deployment frequency

Visual components

- Load Balancer
- API Gateway
- Compute
- Kubernetes
- Database
- Cache
- Queue
- Kafka
- CDN
- Object storage
- Firewall / WAF
- Monitoring
- DNS
- IAM
- AWS and Azure components
- Generic infrastructure components

Component inspector

- Database engine
- Multi-AZ
- Read replicas
- Backups
- Encryption
- Private subnet
- Network configuration
- Other component-specific operational settings

Architecture scorecard

Dimension	Example score
Requirements Coverage	92%
Scalability	88%
Availability	91%
Reliability	83%
Security	76%

Dimension	Example score
Networking	89%
Data Architecture	81%
DR	62%
Observability	74%
CI/CD	86%
Cost Optimization	68%
Operational Complexity	79%
Overall	81%

Review behavior

- Identify concrete design problems rather than only giving a number.
- Examples: single-AZ database, no cache failure strategy, no backup/restore strategy, no multi-region despite DR requirements.
- Separate deterministic violations from semantic AI recommendations.
- Show findings directly on the architecture where possible.

Architecture versioning

V1 62%
V2 74%
V3 86%
V4 91%

Failure injection

- Database unavailable
- Redis down
- Kafka lag
- ALB 5xx
- Availability Zone failure
- Region failure

Trade-off simulator

- Cost vs availability
- Performance vs consistency
- Complexity vs scalability
- Managed vs self-managed
- Kubernetes vs serverless
- Kafka vs SQS
- SQL vs NoSQL

- Multi-region vs cost

Example exercise: architecture costs ■14 lakh/month; reduce to ■10 lakh without violating the availability requirement.

Security review

- Internet exposure
- Network segmentation
- IAM
- Secrets
- Encryption
- TLS
- WAF
- Security groups
- Private subnets
- Least privilege
- Audit logging
- Container security
- Supply-chain security

Architecture pattern library

- API Gateway
- Circuit Breaker
- Retry
- Bulkhead
- Cache-aside
- Queue-based load leveling
- Saga
- Strangler Fig
- CQRS
- Event sourcing
- Blue/Green
- Canary
- Active/Passive
- Active/Active

Each pattern should document the problem, pattern, architecture, when to use, when not to use, trade-offs, real-world example and interview questions.

Phase 5 — Production Incident Simulator & Chaos

Production console

The simulator should feel like an operations console rather than a quiz. The candidate receives partial evidence and must investigate using the tools available.

Example incident

```
Payment API latency: 8.2s
Error rate: 18.4%
CPU: 42%
DB connections: 98%
Redis: healthy
Kafka: lagging
```

Candidate workflow

- Investigate using terminal commands and telemetry.
- Build and test hypotheses.
- Request or discover evidence progressively.
- Identify root cause.
- Mitigate the incident.
- Perform RCA.
- Recommend prevention.
- Communicate status and decisions.

Evaluation

- Investigation order
- Commands selected
- Hypothesis quality
- Evidence usage
- Root-cause identification
- Mitigation quality
- Prevention
- Communication

Chaos / failure injection

```
T+00 Normal
T+02 Redis latency
T+04 Kubernetes node failure
T+06 DB connections 90%
T+08 Kafka lag
T+11 Deployment
T+12 Error rate 15%
```

After architecture design, the candidate can enter a controlled failure simulation and respond under changing conditions.

Phase 6 — Hands-on Labs & Terminal

The product should prove actual operational ability, not only verbal knowledge.

Interactive terminal

- kubectl

- docker
- terraform
- git
- curl
- dig
- nslookup
- ss
- netstat
- journalctl
- top
- systemctl
- aws
- az
- helm
- argocd

Example labs

- Fix CrashLoopBackOff
- Recover Terraform state lock
- Investigate Linux memory pressure
- Troubleshoot service connectivity
- Repair a failed deployment
- Diagnose Kubernetes networking
- Investigate certificate or secret failures

Future sandbox direction

Controlled environments can eventually use Docker, Kubernetes, Terraform and cloud environments. The system should isolate, reset and observe labs so the candidate can make real changes without damaging production infrastructure.

Phase 7 — Observability Studio

- Logs
- Metrics
- Traces
- OpenTelemetry
- Prometheus
- Grafana
- Loki / ELK
- Jaeger / Tempo

Users design observability coverage and troubleshoot simulated failures using telemetry. The goal is to teach the operational relationship between symptoms, metrics, logs, traces and root cause.

Phase 8 — CI/CD & DevSecOps Studio

CI/CD Studio

```
graph TD; Git --> Build; Build --> UnitTest[Unit Test]; UnitTest --> SAST; SAST --> SCA; SCA --> DockerBuild[Docker Build]; DockerBuild --> ImageScan[Image Scan]; ImageScan --> Registry; Registry --> DeployDev[Deploy Dev]; DeployDev --> IntegrationTest[Integration Test]; IntegrationTest --> QA; QA --> Approval; Approval --> Production;
```

- Decide where scans belong.
- Place approvals and production protection.
- Handle secrets securely.
- Design rollback.
- Use canary or progressive delivery.
- Protect production branches/environments.
- Reason about artifact promotion.

DevSecOps

Security is integrated throughout the platform rather than isolated as a separate subject.

- IAM
- RBAC
- Secrets
- KMS
- TLS
- Network security
- WAF
- SAST
- DAST

- SCA
- SBOM
- Container scanning
- Image signing
- Supply-chain security
- Cloud security
- Audit logging

Security incidents

- AWS key committed to Git
- Kubernetes secret exposed
- Vulnerable container image
- Suspicious CloudTrail activity

Practice the sequence: detect → contain → investigate → eradicate → recover → prevent.

Phase 9 — AI Interviewer, Resume & JD Intelligence

AI Interviewer

- Dynamic follow-ups
- Resume-grounded questioning
- JD-grounded questioning
- Technical rounds
- Architecture rounds
- Troubleshooting rounds
- Behavioral rounds
- Eventually voice interviews
- Hide scores during the interview; reveal them afterward.

Resume interrogation

Every resume claim should become a potential interview path.

Resume:
"Implemented Kubernetes"

Possible questions:
→ Why Kubernetes?
→ Explain the architecture.
→ Which CNI?
→ How does ingress work?
→ How did you secure it?
→ How did you monitor it?
→ Hardest production incident?
→ What would you change today?
→ Why this design instead of an alternative?

Goal: every significant resume claim must be technically defensible.

JD Analyzer

Paste a job description and extract technologies, responsibilities and priorities. Generate a personalized preparation plan.

Example:
Kubernetes 91%
AWS 88%
Terraform 82%
Security 61%

- Generate JD-specific interview questions.
- Prioritize preparation by role relevance.
- Compare JD requirements against current readiness.
- Identify missing evidence.

Phase 10 — Behavioral, Communication & Experience Intelligence

Behavioral topics

- Conflict
- Mentoring
- Production mistakes
- Stakeholder management
- Leadership
- Prioritization
- Difficult decisions
- Failure
- Disagreement
- Pressure
- Unrealistic deadlines

Answer frameworks

Answer type	Framework
Technical	Definition → Architecture → How it works → Example → Trade-offs
Troubleshooting	Symptoms → Hypothesis → Evidence → Isolation → Root cause → Remediation → Prevention
Behavioral	Situation → Task → Action → Result → Lesson
Architecture	Requirements → Constraints → Design → Trade-offs → Failure modes → Security → Observability → DR

Communication coach

- Technical accuracy
- Structure

- Clarity
- Conciseness
- Confidence
- Filler words
- Rambling
- Examples
- Trade-off explanation

Eventually support voice interviews with spoken questions, spoken answers, transcription, technical evaluation, communication evaluation, filler-word analysis, pace, confidence and answer structure.

Personal experience / STAR database

- Situation
- Task
- Action
- Result
- Technology
- Problem
- Responsibility
- Challenges
- Metrics
- Lessons

Generate questions from the user's real experience rather than relying only on generic behavioral questions.

Phase 11 — Adaptive Learning & Readiness Engine

Evidence dimensions

- Knowledge
- Practice
- Hands-on
- Troubleshooting
- Architecture
- Security
- Communication
- Confidence
- Incident response

Knowledge vs confidence

Scenario	Interpretation
Knowledge 86%, Confidence 61%	Knows the material but lacks confidence; prioritize verbal practice and interview exposure.
Knowledge 58%, Confidence 89%	Potential overconfidence; require deeper testing and hands-on evidence.

Every wrong answer becomes data

Terraform State

Attempts: 14

Correct: 8

Accuracy: 57%

Confidence: Low

Recommended:

1. 5-minute explanation
2. Infographic
3. Flashcards
4. Scenarios
5. Hands-on
6. Re-test

Readiness levels

Level	Meaning
1	Learner
2	Practitioner
3	Interview Ready
4	Senior Interview Ready
5	Strong Senior
6	Architect

Progression should require evidence thresholds. A user should not reach a higher readiness level based on a single strong quiz result.

Daily adaptive plan — example 90 minutes

- 10 min weak-topic revision
- 15 min flashcards
- 20 min interview questions
- 20 min troubleshooting
- 15 min architecture
- 10 min verbal practice

Tomorrow's allocation should change based on performance.

Phase 12 — Full Interview Day & Emergency Mode

Emergency interview mode

If the interview is tomorrow, prioritize high-value readiness rather than broad deep study.

- Resume interrogation
- JD technologies
- Weak areas
- Common Senior DevOps questions
- Architecture
- Troubleshooting
- Behavioral
- Communication

Full Interview Day

Round	Duration	Focus
1	15 min	Screening
2	30 min	Technical
3	20 min	Troubleshooting
4	30 min	Architecture
5	20 min	Behavioral
6	20 min	Final interviewer

No hints during the simulated interview. Scores and feedback are revealed afterward.

Interview Replay

Save Question → Answer → Follow-up → Answer → Evaluation → Score. Weak topics can later be retested.

Phase 13 — UI/UX Design System & Current Visual Direction

Product brand

NEURAL OPS — DevOps Interview Brain

Brand concept: technical intelligence + engineering precision + focused learning + controlled complexity.

Potential tagline: “Train. Design. Troubleshoot. Interview.” / “From knowledge to production thinking.”

Evolution of visual direction

The product began with a dark-first “Neural DevOps Command Center” direction, then evolved to a lighter, neutral, more engaging interface at the user's request. The current direction keeps the technical precision but improves readability, accessibility and visual warmth.

Current visual personality

- Professional
- Technical

- Intelligent
- Focused
- Modern
- High-performance
- Engineering-first
- Slightly futuristic
- Light and neutral by default

Avoid

- Cartoon education UI
- Excessive neon
- Gaming XP mechanics
- Generic SaaS purple-gradient appearance
- Excessive glassmorphism
- Hacker aesthetic
- Over-animation

Current light visual system

Role	Direction
Background	Very light cool neutral / white
Surfaces	White cards with subtle elevation
Borders	Soft gray-blue borders
Primary	Professional electric blue
Secondary	Controlled AI violet
Success	Emerald
Warning	Amber
Critical	Red
Typography	Modern geometric sans + monospace technical font
Depth	Subtle shadows and elevation

The original dark palette remains a useful alternative theme direction: background #0B0F14, surface #111820, elevated #161E28, border #26313D, primary text #F1F5F9, secondary #94A3B8, blue #3B82F6, violet #8B5CF6, emerald #10B981, amber #F59E0B, red #EF4444, cyan #22D3EE.

Typography

- Primary UI: Inter / Manrope-class modern sans-serif.
- Technical content: JetBrains Mono / Roboto Mono.
- Use monospace only for commands, code, metrics and technical identifiers.

Iconography

The interface should use a consistent professional icon system rather than text glyphs. Material Symbols / MUI Icons and Lucide React are suitable choices.

Motion principles

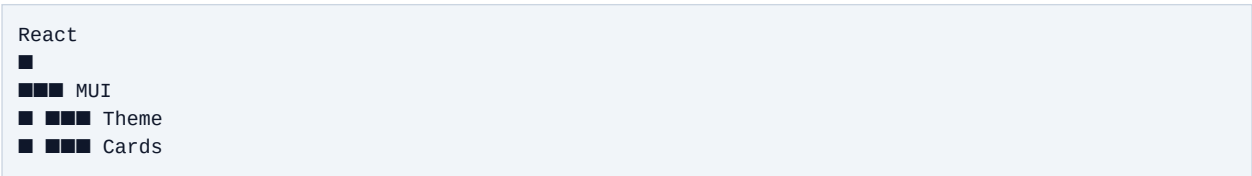
- Progress animations
- Architecture connection animations
- Incident status changes
- Score transitions
- Subtle AI processing states
- Card entrance and hover depth
- Smooth navigation and panel transitions
- Minimal motion during interviews to avoid distraction
- Respect reduced-motion accessibility preferences

Phase 14 — Motion, 3D & Advanced Frontend Stack

The latest agreed direction is to use specialized React libraries where they add functional value rather than making the entire product 3D.

Package	Role	Priority
MUI	Core component/design system	★★★★★
@mui/icons-material	Product icons	★★★★★
Framer Motion	UI transitions and micro-interactions	★★★★★
React Flow	Architecture Studio graph/editor	★★★★★
Three.js	3D rendering foundation	★★★★
React Three Fiber	Declarative React 3D	★★★★
Drei	R3F helpers/components	★★★★
GSAP	Complex timeline animation	★★★
Lucide React	Additional lightweight icons	★★★
MUI X Charts	Readiness/analytics charts	★★★★
MUI X Data Grid	Evidence and analytics tables	★★★
React Spring	Optional physics-based interactions	★★
Spline / react-spline	Optional designer-created 3D scenes	★★

Recommended visual architecture



- ■■■ DataGrid
- ■■■ Charts
- ■■■ Icons
-
- Framer Motion
- ■■■ Page transitions
- ■■■ Score animations
- ■■■ Micro interactions
- ■■■ Interview transitions
-
- React Flow
- ■■■ Architecture Studio
-
- React Three Fiber + Drei
- ■■■ Neural Core
- ■■■ Infrastructure topology
- ■■■ System health
- ■■■ Incident visualization
-
- GSAP
- ■■■ Incident timeline
- ■■■ Chaos simulation
- ■■■ Interview simulation
-
- Lucide React
- Application icons

Where 3D should be used

- Neural Core / AI visualization
- Infrastructure topology
- Architecture depth
- Incident propagation
- System health visualization
- Interactive cloud/service relationships

Do not use 3D everywhere. The product must remain a serious engineering and interview platform.

Architecture Studio UI direction

Use a Figma/cloud-architecture/terminal hybrid: left component palette, center canvas, right inspector, dark or softly tinted engineering grid, nodes, connections, minimap, zoom/pan and health/status indicators. The surrounding application remains light and neutral.

Incident Simulator UI direction

Use an operations-console style with incident severity, impact, service health, telemetry and terminal actions. Motion should communicate changing system state rather than decoration.

AI Interviewer UI direction

Use an interview-room feel rather than a chatbot: AI orb, round timer, response area, subtle processing animation and a restrained visual hierarchy. Scores remain hidden until the interview ends.

Phase 15 — Navigation & Information Architecture

COMMAND
Command Center

LEARN

Knowledge
Documents
Flashcards

PRACTICE
Questions
Incidents
Hands-on Labs

DESIGN
Architecture Studio
Patterns

INTERVIEW
AI Interviewer
Resume Interrogation
JD Analyzer

ANALYTICS
Readiness

Different mental states

Mode	UX feel
Learn	Calm, readable, information-rich
Practice	Focused and compact
Simulate	Immersive and high-pressure
Analyze	Data-rich and diagnostic

Command Center

- Readiness ring
- Skill metrics
- Focus today
- Skill matrix
- Continue training
- Adaptive recommendations
- Recent evidence

Phase 16 — Data Model & Evidence Model

The application should treat every interaction as structured evidence. This makes the readiness score explainable.

Core entities

- User
- Technology
- Document
- Document Chunk
- Source / Provenance

- Topic
- Skill
- Question
- Question Attempt
- Flashcard
- Flashcard Review
- Scenario
- Scenario Node
- Scenario Attempt
- Incident
- Incident Evidence
- Lab
- Lab Attempt
- Architecture Scenario
- Architecture Version
- Architecture Component
- Architecture Finding
- Architecture Score
- Interview Session
- Interview Round
- Interview Question
- Interview Answer
- Evaluation
- Resume Claim
- JD Requirement
- Behavioral Story
- Readiness Snapshot
- Daily Plan

Evidence provenance

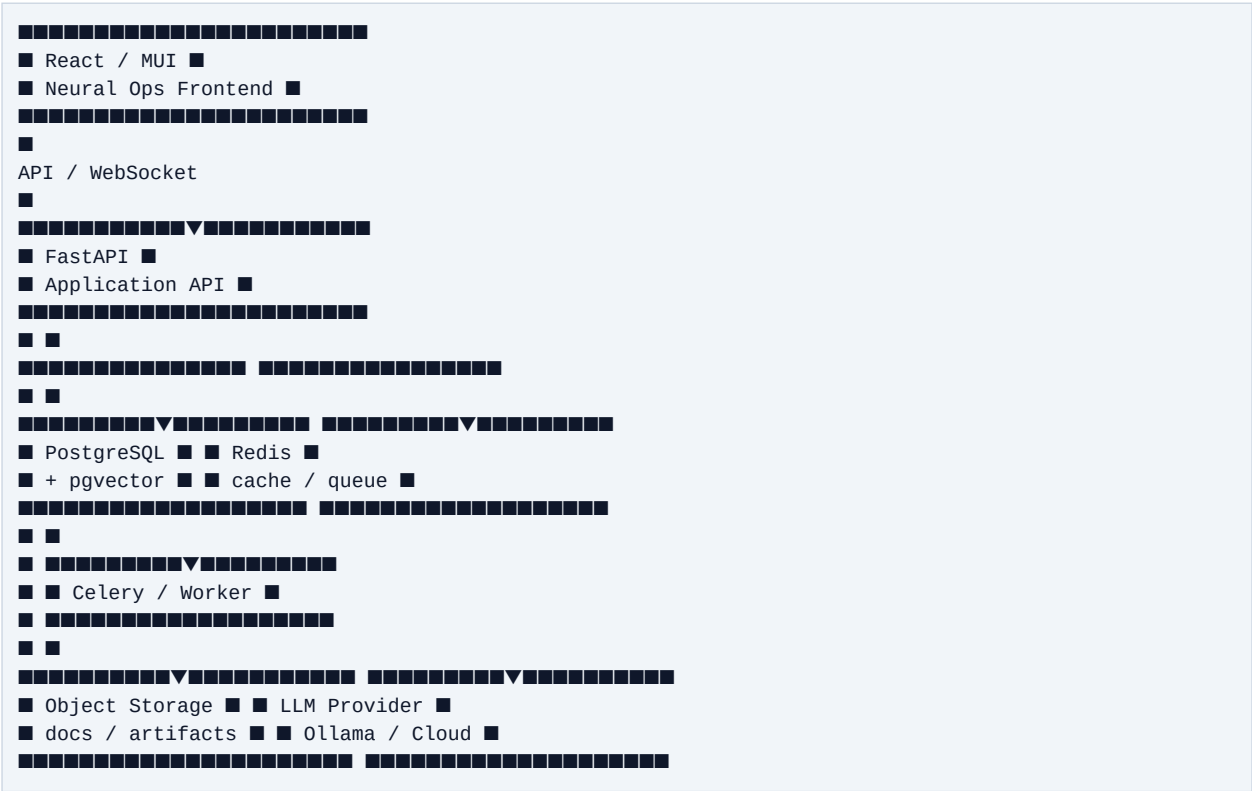
Source type	Meaning
Official documentation	Authoritative external knowledge source
Personal note	User-provided knowledge
Runbook	Operational procedure
AI generated	Machine-created interpretation or content
Interview question	Assessment artifact
Incident	Operational simulation evidence

Source type	Meaning
Lab	Hands-on evidence
Architecture	Design evidence
Interview	Communication and reasoning evidence

Important data principles

- Keep raw evidence.
- Keep evaluation separate from evidence.
- Keep AI reasoning separate from deterministic score calculation.
- Preserve timestamps and attempts.
- Allow replay and re-evaluation.
- Never overwrite historical performance when recalculating readiness.

Phase 17 — Recommended Production Architecture



Operational stack

- Docker for development and packaging.
- Kubernetes as the eventual deployment platform.
- Prometheus + Grafana for observability.
- Object storage for uploaded documents and generated artifacts.
- Background workers for ingestion, embedding, generation, evaluation and lab orchestration.

Security requirements

- Tenant/user isolation.
- Encrypted secrets.
- Strict access controls.
- Audit logging.
- Sandbox isolation for hands-on labs.
- Safe command execution.
- Cloud credential isolation.
- Document access controls.
- Prompt-injection defenses for retrieved documents.
- PII and secret redaction where appropriate.

Phase 18 — Implementation Roadmap

Phase 0 — Product foundation

- Finalize product identity and scope.
- Establish design tokens.
- Define core domain entities.
- Create application shell.
- Create readiness model.

Phase 1 — UI foundation

- React + TypeScript.
- MUI theme.
- Navigation.
- Responsive layout.
- Command Center.
- Knowledge pages.
- Question pages.
- Analytics shell.

Phase 2 — Knowledge platform

- Document upload/import.
- Chunking and indexing.
- PostgreSQL + pgvector.
- Source provenance.
- Reading mode.
- AI contextual assistant.
- Mind maps.

- Infographics.
- Flashcards.

Phase 3 — Question engine

- Question taxonomy.
- Question bank.
- Attempts.
- Evaluation.
- Adaptive difficulty.
- Scenario graph.

Phase 4 — AI layer

- Provider abstraction.
- Ollama integration.
- Optional OpenAI/Anthropic integration.
- RAG pipeline.
- Evaluation pipeline.
- Prompt/version management.
- Guardrails.

Phase 5 — Architecture Studio

- React Flow editor.
- Component palette.
- Inspector.
- Requirements panel.
- Deterministic architecture rules.
- AI architecture review.
- Versioning.
- Scorecards.
- Failure injection.
- Cost estimation.

Phase 6 — Production Simulator

- Incident model.
- Evidence model.
- Operations console.
- Terminal.
- Telemetry.

- Progressive evidence.
- RCA scoring.
- Chaos events.

Phase 7 — Hands-on labs

- Sandbox lifecycle.
- Docker labs.
- Kubernetes labs.
- Terraform labs.
- Linux labs.
- CI/CD labs.
- Cloud labs.

Phase 8 — Interview intelligence

- Resume interrogation.
- JD analyzer.
- Dynamic AI interviewer.
- Behavioral database.
- Interview replay.
- Full interview day.

Phase 9 — Voice & advanced simulation

- Speech-to-text.
- Voice interviewer.
- Communication scoring.
- Advanced incident simulation.
- 3D system visualization.
- Multi-user or team simulations if required.

Phase 19 — Current Prototype Status

The existing prototype has evolved from the original static concept into a richer frontend prototype and now serves as the visual foundation for the production React implementation.

Existing prototype capabilities

- Neural Ops dark-first command-center concept.
- Responsive sidebar/navigation.
- Readiness dashboard and skill matrix.
- Architecture Studio visual canvas.
- Architecture components and inspector.

- Production incident simulator.
- Interactive command console.
- AI Interviewer screen.
- Knowledge graph.
- Documents library.
- Interview questions.
- Readiness analytics.
- Flashcards.
- Hands-on labs.
- Architecture patterns.
- Resume interrogation.
- JD analyzer.
- Interactive navigation.
- Toast interactions.
- Responsive desktop/mobile behavior.

Current visual update

- Light neutral theme.
- More engaging visual hierarchy.
- More icons.
- Material Symbols direction.
- MUI production dependency direction.
- Modern typography.
- Subtle elevation and hover states.
- Animated readiness/progress concepts.
- Neural/3D-inspired visualization.
- Framer Motion direction.
- React Three Fiber + Drei direction.
- React Flow direction for Architecture Studio.
- GSAP direction for complex timelines.

Current prototype limitation

The current artifact is a frontend prototype. Backend persistence, authentication, PostgreSQL/pgvector, Ollama/API LLM integration, deterministic architecture evaluation, real lab infrastructure, real cloud integrations, production observability and full interview persistence are not yet implemented.

Phase 20 — Future Expansion / Advanced Capabilities

- Voice-first interview mode.
- Live terminal co-pilot with controlled hints.

- Real cloud sandbox provisioning.
- Multi-region architecture simulation.
- Automated Terraform validation.
- Kubernetes cluster simulation.
- Incident replay and comparative analysis.
- Architecture benchmark library.
- Company-specific interview packs.
- Role-specific readiness profiles.
- Staff/Architect assessment tracks.
- Interview performance trend analysis.
- Peer or mentor review.
- Team training and enterprise dashboards.
- Custom organization question banks.
- Certification preparation overlays.
- Career progression tracking.
- Automated evidence portfolio.
- AI-generated personalized study curriculum.
- Cross-topic dependency graphs.
- Knowledge decay prediction.
- Interview probability / risk estimation based on evidence.

Appendix A — Complete Technology & Scenario Library

Technology learning domains

- Linux
- Git
- Networking
- AWS
- Azure
- Kubernetes
- Docker
- Terraform
- Helm
- ArgoCD
- Jenkins
- CI/CD
- Ansible
- Python

- PostgreSQL
- Kafka
- Redis
- Prometheus
- Grafana
- OpenTelemetry
- Security / DevSecOps
- Cloud architecture
- Distributed systems
- Observability
- Reliability engineering

Application architecture patterns

- 3-tier
- Monolith
- Microservices
- Event-driven
- Queue-based
- Serverless
- CQRS
- Event sourcing
- Real-time
- Batch
- Streaming
- Distributed systems

Business systems

- E-commerce
- Banking / payment
- SaaS
- Social media
- Food delivery
- Ride sharing
- Logistics
- Healthcare
- IoT
- Media streaming
- Search

- Recommendation systems

Infrastructure/system scenarios

- High availability
- Multi-AZ
- Multi-region
- Active-active
- Active-passive
- Disaster recovery
- Zero downtime
- Blue/Green
- Canary
- GitOps
- IaC

Operational failures

- Database failure
- Network failure
- DNS failure
- Kafka failure
- Redis failure
- Kubernetes failure
- Cloud outage
- Certificate expiry
- Secret expiry
- Deployment failure
- Traffic spike
- Data corruption
- Security breach

Appendix B — Scoring & Evaluation Framework

Question scoring

- Technical correctness
- Completeness
- Depth
- Reasoning
- Trade-offs

- Troubleshooting sequence
- Communication

Troubleshooting scoring

- Problem clarification
- Scope identification
- Timeline/recent-change analysis
- Hypothesis formation
- Evidence gathering
- Isolation
- Root-cause reasoning
- Prioritization
- Mitigation
- Prevention
- Communication

Architecture scoring

- Requirements coverage
- Scalability
- Availability
- Reliability
- Security
- Networking
- Data architecture
- DR
- Observability
- CI/CD
- Cost
- Operational complexity

Interview scoring

- Technical accuracy
- Depth
- Follow-up handling
- Architecture thinking
- Incident response
- Communication
- Confidence

- Structure
- Conciseness
- Trade-off reasoning

Readiness calculation philosophy

Use weighted evidence rather than a single quiz score. Strong readiness should require consistent evidence across multiple modes. Historical evidence should remain available so the system can explain why the score changed.

Appendix C — Product Principles / Non-Negotiables

- 1 Interview readiness is an evidence problem, not a content-consumption problem.
- 2 Documentation is a knowledge source, not the complete learning experience.
- 3 Official sources and AI-generated material must retain distinct provenance.
- 4 Deterministic systems remain authoritative for scoring and readiness thresholds.
- 5 LLMs should support reasoning and generation, not silently define the user's readiness.
- 6 Every major weakness should lead to an actionable next step.
- 7 Every wrong answer should become learning data.
- 8 Hands-on ability must be represented separately from theoretical knowledge.
- 9 Knowledge and confidence must be measured separately.
- 10 Architecture must be evaluated against explicit requirements.
- 11 Troubleshooting should reward evidence-driven investigation and clarifying questions.
- 12 Senior-level preparation must include communication, leadership and behavioral evidence.
- 13 The UI should be engaging but never undermine professional interview preparation.
- 14 3D and animation should communicate system state, relationships or progress—not exist merely as decoration.
- 15 Accessibility and reduced-motion behavior are first-class requirements.
- 16 The product should evolve toward a real React + TypeScript + MUI implementation rather than remain a static HTML prototype.
- 17 Every major feature should contribute measurable evidence to the readiness engine.
- 18 Historical attempts and evaluations should be retained for replay and trend analysis.
- 19 Security and sandbox isolation are mandatory for any real command execution or cloud lab.
- 20 The ultimate outcome is confidence grounded in demonstrated capability.

Final Product Definition

Neural Ops is not a DevOps notes application. It is a continuous interview-readiness system that converts knowledge into evidence, evidence into diagnosis, diagnosis into targeted practice, and practice into measurable Senior DevOps interview readiness.

The intended end state is a platform where a candidate can upload their knowledge sources, resume and target job description; learn and recall the relevant material; troubleshoot realistic production failures; design and defend architectures; operate hands-on environments; practice behavioral stories; undergo dynamic AI interviews; and receive a continuously updated, explainable readiness assessment.

North-star outcome: When the user asks “Am I ready for a Senior DevOps interview today?”, Neural Ops should be able to answer with evidence, identify exactly where the candidate is likely to fail, and produce the highest-value actions to close those gaps.